

Tvorba internetových aplikací

1. Základy internetových aplikací.

- HTTP, HTML, CSS, SVG, JavaScript.
 - HTTP
 - Protokol, akým komunikujú klient (user agent) a server. Ma requesty (GET, POST, ...) a responsy (100, 200, 300, 400, 500)
 - HTML
 - Jazyk pre dokumenty ktoré sa majú zobrazovať prehliadacom.
 - CSS
 - Jazyk pre stylovanie dokumentov v HTML.
 - SVG
 - Formát vektorových obrázkov založený na XML.
 - JavaScript
 - Programovací jazyk používaný hlavne na strane klienta.
- Oddelenie prezentácie, obsahu a kódu, šablóny.
 - Rozdelenie na .html, .css a .js súbory.
 - Šablóna je html súbor, ktorý sa vyplní dátami zo servera (teda je na serveri a vyplnená sa pošle na klienta).
- JavaScript transpilery.
 - Transpiler: prekladá vyšší jazyk do iného vyššieho jazyka
 - {TypeScript, CoffeeScript, PureScript, ...} -> Javascript
 - Novšie verzie javascriptu -> javascript
- jQuery – výhody a nevýhody.
 - JS knižnica na zjednodušovanie práce s DOM, eventami, animáciami, AJAX
 - Výhody:
 - Open-source
 - Celkom mála
 - Browser support
 - Nevýhody
 - Celkom pomaly (napríklad oproti natívnym selektorom)
- Prechod od statických stránok k dynamickým aplikáciám (cookies, sessions, upload, dynamické generovanie obsahu – databáza).
 - Cookies
 - Data vytvorené asi serverom a uložené v browseri
 - Napríklad pre autentifikáciu
 - Trackovanie
 - Session
 - Pri nadviazaní spojenia so serverom server vygeneruje SessionID a to vráti klientovi. Ten si to uloží asi do cookie a potom pošle SessionID s každým ďalším requestom. Už sa teda nemusí autentifikovať a navyše server môže zbierať informácie u seba a neukladať všetko u klienta.
 - Upload

- Form je sandboxed, takže neda sa priamo napísať meno súboru. Da sa iba vybrať cez OS, cez nejaké UI čo poskytne desktop environment.
 - Folder upload je divný
- Dynamické generovanie obsahu - databáza
 - Templaty

2. JavaScript OOP.

- Objektovo orientované programovanie založené na prototypoch (rozdiely oproti OOP založenému na triedach).
 - OO:
 - class je typ
 - objekty sú instance triedy
 - hierarchia je podľa dedenia
 - premenné a metódy objektu sa určujú podľa inheritance chain
 - class špecifikuje finálny set premenných a metód (nedajú sa pridať v runtime)
 - Prototypy:
 - nie je tu class, iba konštruktorové funkcie
 - objekty sú dictionaries
 - hierarchia sa robí tak, že konštruktorovej funkcii priradíme objekt ako prototyp s nou asociovaný
 - premenné a metódy objektu sa určujú podľa prototype chain
 - konštruktorová funkcia špecifikuje iba iníciaľnú množinu premenných a metód, dajú sa pridať v runtime
- Rozdiel medzi `__proto__` a prototypovými položkami objektov.
 - **`__proto__`** je property každého objektu, ukazuje na prototypový objekt
 - **prototyp** je objekt, z ktorého sa vytvorí **`__proto__`** objektu, keď použijeme **`new`**.
 - Teda **prototyp** je dostupný iba v konštruktorových funkciách, nie na objektoch samotných
 - **`__proto__`** sa využíva v prototypovom lookup chaine
- Manipulácia s `this` pomocou `call`, `apply` a `bind` a jej využitie.
 - **`this`** v globálnom kontexte je **`window`**, ale pri strict mode je to **`undefined`**
 - rovnako vo funkcii ktorá nie je súčasťou objektu
 - V konštruktorovej funkcii je **`this`** nabitované na aktuálne vytváraný objekt
 - `call`, `apply`:
 - Každá funkcia má `.call` a `.apply` (z `Function.prototype`)
 - `call(thisArg, arg1, ..., argn)`
 - `apply(thisArg, [arg1, ..., argn])`
 - `add.call(o, 5, 7)`
 - `add.apply(o, [5, 7])`
 - `bind`:

- Vrať novú funkciu, v ktorej je `this` nabitované na to, čo chceme bez ohľadu na to, ako sa zavola
 - `fun.bind(thisArg, arg1, ..., argn)`
 - Shortcuts:
 - `Function.prototype.apply.bind`
 - Potom nemusíme používať `apply` pri volaní
- Privátne položky a metódy v JavaScript objektoch a ich súvis s closures.
 - Objekty nemôžu mať private položky a metódy
 - Ale dajú sa pridať pomocou closures (příklad s function `Container(private_value)`)

3. JavaScript.

- Typované polia (`buffer`, `view`, `DataView`).
 - `buffer`:
 - Chunk dát, nedať sa k nim prístupovať, nemajú typ.
 - `var buf = new ArrayBuffer(16);` // 16 bytov
 - Inicializácia sa dá aj:
 - `var byteArray = new Uint8Array([1, 2, 3, 4]);`
 - Underlying `ArrayBuffer` sa dá získať cez `byteArray.buffer`
 - Konverzia na normálne pole:
 - `Array.apply([], byteArray);`
 - `view`:
 - Dáva typ dátam, môže mať `starting offset`
 - `var int32Vw = new Int32Array(buf);` // `int32Vw = {0: 0, 1: 0, 2: 0, 3: 0}`
 - Dá sa tu už spraviť `int32V2[1] *= 3;`
 - Na rovnaký `buffer` môžeme mať aj iný `view` (bude meniť rovnaký `buffer`)
 - `var int16Vw = new Int16Array(buf, 2, 6);` // `buffer`, `offset (2B)`, `pocet prvkov`
 - `DataView`:
 - Pri `view` musia byť dáta zarovnané (ináč nemôžeme spraviť `Int16Array` z `bufferu` párnej dĺžky tak, že mu dáme `offset 1`).
 - `DataViewu` môžeme dať ľubovoľný `buffer`, ľubovoľný `offset` a poprosiť si napríklad 16-bytový `int` a on nám ho z toho `bufferu` vytiahne
 - `dataView.getInt16(byteOffset, bigEndian)`
- Strict mode.
 - "use strict" podľa scope kde to chceme
 - Zakazuje robiť globálne premenne chybou (let `misspelledVariable`; `mispeledVariable = 16;` // toto dá chybu, nevytvorí novú premennú).
 - Assignment do non-writable property dá error
 - Zakazuje syntax ktorá sa pravdepodobne bude využívať v budúcich verziách ES
- Funkcie s premenlivým počtom argumentov.
 - Hack: `function f(...args)`

- Vo funkcii sa vieme odvolavat na **arguments**, čo je zoznam vsetkych argumentov
- Block vs. function scoping (t.j. let vs. var).
 - var: function scoping
 - Ak je niekde **A; var x = 10; B;**, tak to zmeni na **var x; A; x = 10; B**
 - Cize vytiahne definicie (bez priradeni) premennych na zaciatok
 - Aj ked mame vo funkcii **var x = 10;** a potom mame v tej funkcii if, v ktorom spravime **var x = 20;**, tak to bude ta ista premenna, nebude to zatienene.
 - let: block scoping
 - Klasicky block scoping, rozumny
- Reťazenie (kaskádovanie) metód.
 - Setter vracia objekt samotny, takže sa na to da hned volat dalsia metoda

4. Canvas element a Scalable Vector Graphics (SVG).

- Porovnanie <canvas> elementu a SVG (nezávislosť na rozlíšení, DOM, CSS, udalosti, odkazy, ...).
 - svg je nezávisle na rozlíšení
 - DOM:
 - Pri svg sa daju inspectovat jednotlivé elementy a da sa nimi programovo manipulovat (to sa pri canvase neda) => animacie, event handlers, ...
 - svg je rychlejsie ako canvas pre veľké obrázky
 - svg nerobi zbytočne HTTP requests (oproti img)
 - Pre komplexné obrázky (veľa častí) je lepší canvas a vyrenderovat to a prípadne odtiaľ vyexportovat do bitmapového obrázku
 - Na svg sa nema pouzivat CSS cez style atribut (ale rovno pisat fill=..., atd.)
 - Svg suradnice su nekonečne, my si urcime viewport. Canvas ma fixnú výšku a šírku.
 - CSS sa na canvas aplikuje normalne, ale to čo je na canvase sa tým neda ovplyvniť. Pri SVG vieme aplikovat CSS na jednotlivé časti.
- Použitie <g> elementu (zoskupovanie), <use> elementu (znovupoužitie) a <def> elementu (definície bez zobrazenia) pri SVG.
 - <g>:
 - Sluzi na zgrupovanie súvisiacich častí SVG
 - Moze mať vlastné id a vlastný title pre text-based aplikacie
 - <use>:
 - Ked mame napríklad <g id="domcek">...</g>, tak potom mozeme inde spravit <use xlink:href="#domcek" x=100 y=200 /> na vykreslenie toho istého domčka na súradniciach 100, 200.
 - <defs>:
 - Tu si mozeme definovat prvky na re-use a potom ich neskor pouzivat pomocou <use>. Teda napríklad ten domček tu definujeme a on sa teda

nevykresli a potom ho mozeme vykreslit vsade kde chceme pomocou `<use>`.

- Transformácia súradnicového systému, gradienty, zarovnanie textu na cestu.
 - Canvas:
 - Všetky potrebné transformácie na canvas (translacia, skalovanie, rotácia, skewing) sa dajú robiť pomocou matice 3x3:
$$\begin{pmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}$$
 - Skladanie transformácií pomocou násobenia matic
 - Gradienty:
 - Lineárny gradient (od bodu [x0, y0] po bod [x1, y1])
 - Radialný gradient (dame mu startovací a konečný kruh a medzi nimi to spraví)
 - `addColorStop(stop, color)` ze kde (od 0 do 1) má byť konkrétna farba. Ono to potom samo spraví gradienty medzi tými farbami v color stopoch.
 - Zarovnanie textu na cestu (to asi nejde bez JS na cestu)
 - `ctx.textAlign = {left, start, center, end, right}`
 - `ctx.textBaseline = {alphabetic, top, middle, bottom, ...}`
 - SVG:
 - Máme transform: `translate` / `rotate` / `scale` / `skewX` / `skewY`
 - Gradienty v defs môžeme definovať a potom používať (`linearGradient`, `radialGradient`)
 - Zarovnanie textu na cestu:
 - V `<defs>` si definujeme `<path>` a dame mu id. Potom vieme spraviť:
 - `<text>`

```
<textPath xlink:href="#cesta_z_defs">
  Nejaký text na ceste
</textPath>
</text>
```
 - Data URL formát a `<img>` element
 - `canvas.toDataURL("image/png")` nám dá url obrázku
 - Data URL formát:
 - `data:[MIME type];charset=<charset>;base64,<encoded_data>`
 - `"..."`
 - V `img` sa dá nastaviť ako **`myImg.src = "..."`**

5. AJAX a WebSockets.

- Princíp AJAX, XMLHttpRequest, výhody a nevýhody.
 - AJAX:

- Posleme request na server a dostaneme response bez refreshovania stránky. Podľa toho sa môže zmeniť iba časť stránky.
 - Už sa nepoužíva moc XML, ale JSON
- XMLHttpRequest:
 - Takto sa robil AJAX bez knižníc.
 - `var xhr = new XMLHttpRequest();`
 - Nastavil sa callback cez `xhr.onreadystatechange = function () {...}`
 - Inicializoval sa ten request cez `xhr.open(method, url, async=true)`
 - Poslal sa cez `xhr.send();`
- Vyhody:
 - Lepšia UX
 - Menej prenášaných dát -> vyššia rýchlosť
- Nevýhody:
 - Zložitejšia implementácia -> viac priestoru na útoky
 - Treba byť pripravený na používateľov, čo majú zakázaný JS
- Notifikácie od servera pred WebSocketmi (Polling, Long Polling, Streaming).
 - Polling:
 - Klient sa pravidelne pýta servera, či nemá nové dáta
 - Long polling:
 - Klient pošle request a server ho drží otvorený, kým nemá nové dáta. Keď má dáta, tak pošle response. Teda sa neposiela zbytočne veľa requestov od klienta. Iba keď vytimeoutuje, tak musí klient poslať nový.
 - Streaming:
 - Server nikdy neukončí response. Nastaví nejaký "chunked" flag, čo znamená, že klient prijme dáta, ale request tým nie je ukončený a čaká sa na ďalší chunk. Toto môže byť spomalené nejakými firewallmi a proxies, ktoré sa môžu snažiť bufferovať tie chunky a poslať ich potom naraz, čo nechceme.
- Výhody WebSocketov (šetrí šírku pásma, CPU, jednoduché API, súčasť HTML5).
 - `var ws = new WebSocket("ws://echo.websocket.org/echo");`
 - `ws.onopen = function(e) { console.log("Connection open..."); };`
 - `ws.onmessage = function(e) { console.log(e.data); }`
 - `ws.close(code=1000, reason="Closing normally");`
 - readyState:
 - CONNECTING, OPEN, CLOSING, CLOSED

6. Bezpečnosť internetových aplikácií

- HTTPS, identifikácia a autentifikácia používateľov.
 - HTTPS:
 - Pre autentifikáciu servera a zariadenie dôveryhodnosti a integrity prenášaných dát

- TLS handshake (SYN, SYN ACK, ACK, ClientHello, ServerHello+Certificate, ClientKeyExchange+ClientCipherSpec, ChangeCipherSpec)
 - Dohodnu verziu TLS
 - Dohodnu cipher suite
 - Dohodnu symetricky kluc
 - Identifikacia:
 - Najcastejsie username / email
 - Autentifikacia:
 - Najcastejsie heslo
 - Session hijacking, session riding.
 - Session hijacking:
 - Utocnik odchyti SessionID
 - Odpocutim komunikacie
 - Z refferer headera pri prechode na inu stranku
 - Z cookies na pocitaci
 - Alebo tipne SessionID, ak sa generuje sekvenčne
 - Obrana:
 - SessionID bude okrem nahodnej zlozky obsahovat aj hash IP adresy pouzivatela, servera, User-Agent info atd. Potom sa neda jednoducho reusnut.
 - Session riding (= CSRF):
 - Utocnik tu nevie SessionID, ale donuti pouzivatela spravit request, aky chce utocnik.
 - Napriklad posle email s linkom, ktorý vykonava nejaku konkretnu akciu na cielovej webstranke. Pouzivatel klikne na ten link a s nim jeho browser automaticky posle cookies (so SessionID) a teda request prejde.
 - Alebo cez , alebo da utocnik link do diskusie
 - Obrana:
 - Nepouzivat cookies na ukladanie SessionID
 - Dat SessionID priamo do URL
 - CSRF token parameter sa prida do kazdej action URL (server vygeneruje s kazdou strankou)
 - URL tampering, forceful browsing.
 - URL tampering:
 - Zmena URL pre request
 - Napriklad ?id=X spravi ?id=X+1
 - Forceful browsing:
 - Utocnik skusi ist na tipnutu podstranku, napriklad domena.com/admin.php, alebo na domena.com/abc.php.old
 - HTML injection, SQL injection.
 - HTML injection:
 - CSRF, napriklad utocnik da URL do diskusie, alebo prida obrazok ktorý bude mat ako src nejaku URL co je zla

- SQL injection:
 - `SELECT * FROM users WHERE name=" OR 1=1;`
- White-list vs. black-list.
 - Blacklist:
 - Zablockuju sa iba zname zle veci
 - Pouzivatelia teda maju k dispozicii vsetko, co chcu (a nemusia ziadat o pridanie niecoho do whitelistu)
 - Vacsia volnost ohladom bezpecnosti
 - Whitelist:
 - Otravnejsie pre pouzivatelov, ale menej priestoru na utoky